

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Higher Institute of Sciences



Projet de fin d'étude

En : Mathématiques et Informatique

Spécialité : Ingénierie de Systèmes d'Information & Logiciels

Intitulé :

Vectorization Automatique d'Images Matricielles par Deep Learning

Encadré par :

ABDELLAHOUM Hamza

Présenté par :

BITAM Amir

AITHAMOUDI Zakaria

Membres du Jury :

????

????

Soutenu le :

??/07/2026

Année universitaire : 2025/2026

Remerciements

Nous tenons à exprimer notre profonde gratitude envers Allah, le Tout-Puissant, pour nous avoir accordé la force, la patience et la persévérance nécessaires pour mener à bien ce projet de fin d'étude.

Nous remercions sincèrement notre encadreur **ABDELLAHOU** **Hamza** pour sa précieuse guidance, son soutien constant et ses précieux conseils tout au long de ce projet.

Nos vifs remerciements s'adressent également aux membres du jury **????** et **????** pour avoir accepté d'évaluer notre travail.

Dédicace

Résumé

Nous proposons dans le cadre de notre projet de fin d'études un modèle d'IA qui traite le zoom et le dézoom dans les images numériques. En zoomant sur une image de taille donnée, sa qualité est souvent altérée et l'image peut devenir pixélisée. Notre travail consiste à améliorer ces images en augmentant leur qualité.

En ce qui concerne le dézoom, en réduisant les dimensions de l'image, il faut minimiser le nombre de pixels tout en préservant l'esthétique de l'image.

Nous avons créé un modèle CNN avec une architecture adaptée, piloté par des métaheuristiques (algorithme génétique) permettant de sélectionner les paramètres optimaux afin d'obtenir de bonnes performances avec un nombre minimal de paramètres.

Mots-clés : Redimensionnement d'images, Super Résolution, Réseaux de Neurones Convolutifs (CNN), Algorithme Génétique, Métaheuristiques, PSNR, SSIM.

Abstract

Keywords : Image Resizing, Super Resolution, Convolutional Neural Networks (CNN), Genetic Algorithm, Metaheuristics, PSNR, SSIM.

Liste des Acronymes

XML	eXtensible Markup Language
SVG	Scalable Vector Graphics
GA	Genetic Algorithm
CNN	Convolutional Neural Networks
PSNR	Peak Signal to Noise Ratio
HR	High Resolution
LR	Low Resolution
SSIM	Structural SIMilarity
SRCNN	Super Resolution Convolutional Neural Network
SRGAN	Super Resolution Generative Adversarial Network
MAE	Mean Absolute Error
MSE	Mean Squared Error
ReLU	Rectified Linear Unit
ResNet	Residual Network
SE	Squeeze-and-Excitation
CAR	Content Adaptive Resampler
FARF	Feature Augmented Random Forest
DRDN	Deep Residual Dense Network
EDSAN	Enhanced Dense Space Attention Network
PPI	Pixels Per Inch
ViT	Vision Transformer
UDF	Unsigned Distance guided Focal loss

Table des matières

Remerciements	i
Dédicace	ii
Résumé	iii
Abstract	iv
Liste des Acronymes	v
Introduction Générale	2
1 État de l'Art	3
1.1 Introduction	3
1.2 La vectorisation d'images	3
1.2.1 Images matricielles	3
A. Pixels	3
1.2.2 Images vectorielles	4
A. Courbes de Bézier	5
B. Format SVG	5
1.3 Apprentissage automatique et apprentissage profond	5
1.3.1 Apprentissage automatique et apprentissage auto-supervisé . . .	6
1.3.2 Apprentissage profond	6
A. Réseaux de neurones	7
B. Transformers	7
C. Vision Transformers	8
1.4 Mécanismes d'attention	9
1.4.1 Self-attention	10
1.4.2 Cross-attention	11
1.5 Concepts utilisés dans la vectorisation par apprentissage profond	11
1.5.1 Superpixels	12
1.5.2 Rendu différentiable	12
1.5.3 Fonctions de perte	12
1.5.4 Apprentissage coarse-to-fine	13
1.6 Évaluation des résultats	13
1.6.1 MSE	13
1.6.2 PSNR	14

1.6.3	SSIM	14
1.6.4	LPIPS	14
1.6.5	Temps d'inférence	15
1.7	Travaux connexes	15
1.7.1	Approches algorithmiques	15
A.	Potrace	15
1.7.2	Approches basées sur l'optimisation	16
A.	DiffVG	16
B.	LIVE	16
1.7.3	Approches basées sur l'apprentissage profond	16
A.	Im2Vec	17
B.	SuperSVG	17
1.8	Synthèse comparative des approches	18
1.9	Conclusion	19
2	Approche Proposée	20
2.1	Introduction	20
2.2	Dataset et prétraitement	20
2.2.1	Le prétraitement des données	20
A.	Former les paires d'images	20
B.	Augmentation de données	20
2.3	Méthode proposée	20
2.3.1	Définir les hyperparamètres évalués par les métaheuristiques	20
A.	L'Algorithme Génétique	20
B.	Les paramètres à optimiser	20
C.	Évaluation d'une solution	20
2.3.2	Architecture	20
A.	Sur-échantillonnage (Upsampling)	20
B.	Sous-échantillonnage (Downsampling)	20
2.4	Conclusion	20
3	Expérimentations et Résultats	21
3.1	Introduction	21
3.2	Environnement de développement	21
3.2.1	Hardware	21
3.2.2	Software	21
3.3	Dataset utilisé	21
3.4	Prétraitement	21
3.5	Évaluation	21
3.5.1	Évaluation quantitative	21

3.5.2	Fonction de perte	21
3.6	Paramétrage de l'Algorithme Génétique	21
3.7	Sous-échantillonnage	22
3.7.1	Hyperparamètres avec la métaheuristique (GA)	22
3.7.2	Résultats	22
A.	Résultats quantitatifs	22
B.	Résultats qualitatifs	22
3.8	Sur-échantillonnage	22
3.8.1	Hyperparamètres	22
3.8.2	Résultats	23
A.	Résultats quantitatifs	23
B.	Résultats qualitatifs	23
3.9	Conclusion	23
Conclusion Générale		24
Bibliographie		25

Table des figures

1.1	Comparaison entre une image matricielle (a) et une image vectorielle (b).	4
1.2	Exemple du contenu d'un fichier .svg	5
1.3	Structure simplifiée d'un encodeur Transformer	8
1.4	Architecture générale d'un Vision Transformer	8
1.5	Attention par produit scalaire mise à l'échelle	10

Liste des tableaux

1.1	Synthèse comparative des principales approches de vectorisation étudiées.	18
3.1	Spécifications de la machine Kaggle	21
3.2	Valeurs du PSNR en fonction du nombre de générations	22
3.3	Résultats du downsampling (Set5) — PSNR / SSIM	22
3.4	Comparaison quantitative sur Set5 (PSNR en dB)	23

Introduction Générale

Le traitement d'images est un domaine de recherche important qui trouve des applications dans divers domaines tels que la médecine, la surveillance, la sécurité, la robotique, l'automobile, etc.

L'un des problèmes les plus courants dans ce domaine est la qualité de l'image qui peut être altérée lorsqu'elle est agrandie ou réduite en taille.

Ce mémoire est organisé comme suit :

- **Chapitre 1** : État de l'art — revue de littérature sur le redimensionnement d'images et les approches existantes.
- **Chapitre 2** : Approche proposée — architecture des modèles CNN et sélection des hyperparamètres par méta heuristiques.
- **Chapitre 3** : Expérimentations et Résultats — évaluation quantitative et qualitative avec comparaisons.

Chapitre 1

État de l'Art

1.1 Introduction

Dans ce chapitre, nous exposerons d'abord les concepts clés à la vectorisation d'images, à l'apprentissage automatique, et les architectures utilisées tout au long de ce travail, ceci nous permettra de mieux appréhender ensuite les travaux effectués sur le sujet.

1.2 La vectorisation d'images

Dans cette section nous tenterons de définir les termes et concepts liés à la transformation d'images matricielles en images vectorielles, dite, vectorisation d'images. En soulignant notamment la différence entre les images dites matricielles et vectorielles. En opposant ainsi les deux types d'images nous pourrions mieux cerner la motivation derrière ce travail et pourquoi la vectorisation d'image constitue un exercice complexe de la vision par ordinateur.

1.2.1 Images matricielles

Les images matricielles peuvent être définies comme des fonctions bidimensionnelles $f(x, y)$, où x et y représentent des coordonnées spatiales planes et où la valeur de f associée à chaque paire de coordonnées correspond à l'intensité ou à la couleur du point considéré. Plus simplement, elles peuvent être représentées comme une grille de pixels organisée selon une largeur et une hauteur données [1]. Les images matricielles présentent toutefois une limite lors de leur agrandissement. En effet, comme elles sont constituées d'un nombre fixe de pixels, une augmentation importante de leurs dimensions peut rendre ces derniers visibles et produire un effet de pixellisation. La qualité de l'image se dégrade alors progressivement à mesure que le facteur d'agrandissement augmente.

A. Pixels

Un pixel, contraction de *picture element*, représente l'unité élémentaire d'une image numérique. Il correspond à un point de la grille matricielle et possède une position, définie par ses coordonnées spatiales, ainsi qu'une valeur décrivant son intensité ou sa couleur. Dans une image en niveaux de gris, cette valeur correspond généralement à une intensité

lumineuse, tandis que dans une image couleur elle est souvent représentée par plusieurs composantes, par exemple rouge, vert et bleu. Ainsi, une image matricielle peut être comprise comme un ensemble fini de pixels organisés selon une largeur et une hauteur données [1]. Cette représentation rend le contenu directement dépendant de la résolution : lorsqu'une image est fortement agrandie, les pixels deviennent progressivement visibles, ce qui produit l'effet de pixellisation. Ainsi, le pixel constitue l'unité élémentaire de l'image matricielle, mais il impose également une granularité fixe qui limite la conservation des détails lors des changements d'échelle.

1.2.2 Images vectorielles

Les images vectorielles sont décrites par un ensemble de formes géométriques individuelles, telles que des cercles, des segments de droite ou des courbes de Bézier. Chacune de ces formes est enregistrée avec les paramètres mathématiques et les attributs qui la définissent, notamment ses points d'ancrage, sa couleur de remplissage ou l'épaisseur de son contour. Contrairement aux images matricielles, dont le nombre de pixels est fixe, les images vectorielles peuvent être agrandies sans perte de qualité, car leur rendu est recalculé pour chaque résolution [2]. Comme l'illustre la figure 1.1, l'image matricielle présentée en (a) devient pixellisée lors de son agrandissement, tandis que l'image vectorielle présentée en (b) conserve des contours nets grâce à sa définition mathématique. Les images vectorielles sont donc largement utilisées dans les domaines du graphisme et de la conception. Cependant, la transformation d'une image matricielle en une image vectorielle reste complexe en raison de la différence entre leurs modes de représentation.

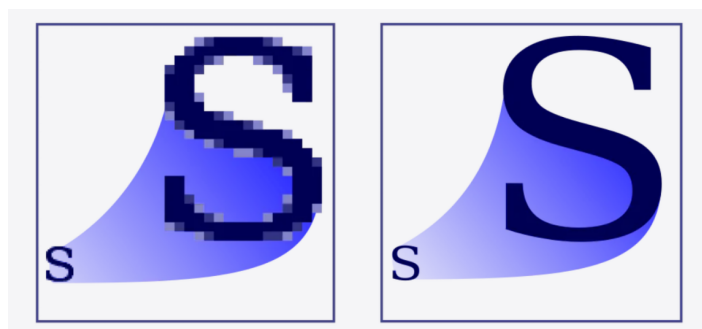


FIGURE 1.1 – Comparaison entre une image matricielle (a) et une image vectorielle (b).

Afin de mieux comprendre la manière dont une image vectorielle est construite et enregistrée, il convient de présenter deux éléments essentiels. Les courbes de Bézier permettent de représenter des contours et des formes complexes à partir d'un nombre limité de points de contrôle, tandis que le format SVG fournit une structure permettant de décrire et de stocker ces primitives vectorielles.

A. Courbes de Bézier

Une courbe de Bézier est définie comme une fonction polynomiale dont la forme est dictée par la position de points de contrôle. La courbe passe par le premier et le dernier point (points d'ancrage), tandis que les points intermédiaires agissent comme des "aimants" qui tirent la courbe vers eux [2].

Les courbes de Bézier font partie des formes géométriques individuelles qui composent les images vectorielles et se distinguent par le faible nombre de données numériques nécessaires à la représentation de formes complexes.

B. Format SVG

SVG (Scalable Vector Graphics) est un format basé sur XML comportant des directives pour la définition de graphiques 2D[3]. Stockées dans des fichiers textes rendant les SVG compressible et indexable, ces directives SVG supportent aussi bien les formes géométriques vectorielle que les images et le texte. Pour les raisons précédentes ce sera ce format qui sera préféré pour la sortie du modèle.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN" "http://
www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">
<svg width="391" height="391" viewBox="-70.5 -70.5 391 391"
xmlns="http://www.w3.org/2000/svg" xmlns:xlink="http://
www.w3.org/1999/xlink">
<rect fill="#fff" stroke="#000" x="-70" y="-70" width="390"
height="390"/>
<g opacity="0.8">
  <rect x="25" y="25" width="200" height="200" fill="lime"
stroke-width="4" stroke="pink" />
  <circle cx="125" cy="125" r="75" fill="orange" />
  <polyline points="50,150 50,200 200,200 200,100" stroke="red"
stroke-width="4" fill="none" />
  <line x1="50" y1="50" x2="200" y2="200" stroke="blue" stroke-
width="4" />
</g>
</svg>
```

FIGURE 1.2 – Exemple du contenu d'un fichier .svg

1.3 Apprentissage automatique et apprentissage profond

Cette section présente les notions d'apprentissage automatique et d'apprentissage profond nécessaires à la compréhension des approches modernes de vectorisation d'images. Elle introduit d'abord les principaux modes d'apprentissage, notamment l'apprentissage auto-supervisé, puis décrit les architectures profondes utilisées pour extraire et traiter les caractéristiques visuelles, en particulier les réseaux de neurones, les Transformers et les Vision Transformers.

1.3.1 Apprentissage automatique et apprentissage auto-supervisé

L'apprentissage automatique est une discipline du domaine de l'intelligence artificielle dans laquelle, contrairement à l'approche programmatique classique, la machine ne reçoit pas une suite d'instructions détaillées pour résoudre une tâche donnée, car ces instructions seraient souvent trop complexes à définir explicitement [4]. Le principe consiste plutôt à apprendre à partir d'un ensemble de données afin d'améliorer progressivement une performance P sur une tâche T à partir d'une expérience E , selon la définition proposée par Mitchell [5].

Selon la nature des données et du signal de supervision disponibles, plusieurs modes d'apprentissage peuvent être distingués. Dans l'apprentissage supervisé, chaque exemple d'entrée est associé à une cible ou à une étiquette attendue. À l'inverse, dans l'apprentissage non supervisé, le modèle cherche à découvrir des structures présentes dans les données, telles que des groupes d'exemples similaires ou des représentations de plus faible dimension [6].

Cependant, dans le contexte de la vision par ordinateur moderne, ces deux catégories ne suffisent pas toujours à expliquer l'intérêt des modèles pré-entraînés utilisés pour l'extraction de caractéristiques visuelles. En effet, l'annotation manuelle de grands ensembles d'images est une tâche coûteuse et parfois difficile à généraliser à tous les domaines. C'est dans ce cadre que l'apprentissage auto-supervisé prend une place importante. Cette approche exploite des données non étiquetées tout en générant automatiquement un signal de supervision à partir des données elles-mêmes, par exemple en demandant au modèle de prédire une partie manquante, une transformation appliquée ou une relation entre différentes vues d'une même image [7]. Elle permet ainsi d'apprendre des représentations visuelles générales à partir de grandes collections d'images sans annotations humaines explicites, représentations qui peuvent ensuite être transférées vers d'autres tâches de vision par ordinateur [4, 7]. Dans le cadre de la vectorisation d'images, cet aspect est particulièrement pertinent, car l'objectif n'est pas seulement de classer une image, mais d'en extraire une représentation riche de sa structure, de ses régions et de ses détails visuels, afin de faciliter ensuite la prédiction de primitives vectorielles.

1.3.2 Apprentissage profond

L'apprentissage profond est une sous-branche de l'apprentissage automatique fondée principalement sur les réseaux de neurones profonds. Ces modèles sont constitués de plusieurs couches de neurones artificiels capables d'apprendre progressivement des représentations de plus en plus abstraites à partir des données. Cette approche s'est fortement développée grâce à l'augmentation de la puissance de calcul, à la disponibilité de grandes quantités de données et aux progrès réalisés dans l'entraînement des

réseaux profonds [4]. Dans le domaine du traitement d'images, les réseaux de neurones convolutionnels (CNN) ont longtemps constitué l'une des architectures dominantes, grâce à leur capacité à exploiter la structure spatiale locale des images et à extraire automatiquement des caractéristiques visuelles telles que les contours, les formes ou les textures [8, 1]. Cependant, les architectures récentes fondées sur l'attention, en particulier les Transformers et les Vision Transformers, ont progressivement élargi cette approche en permettant de modéliser des relations plus globales entre les différentes régions d'une image.

A. Réseaux de neurones

Les réseaux de neurones artificiels sont des modèles d'apprentissage automatique inspirés, de manière simplifiée, du fonctionnement des neurones biologiques. Ils sont constitués d'unités de calcul appelées neurones artificiels, organisées en couches et reliées entre elles par des connexions pondérées. L'apprentissage consiste à ajuster ces poids à partir des données d'entraînement afin que le réseau puisse produire une sortie correcte pour une tâche donnée [4]. Dans le domaine du traitement d'images, ces modèles sont utilisés pour apprendre automatiquement des fonctions de décision à partir d'exemples, notamment dans les problèmes de classification, de reconnaissance de formes et de reconstruction [1].

B. Transformers

Les Transformers sont une architecture de réseaux de neurones introduite initialement pour le traitement des séquences, notamment en traduction automatique. Contrairement aux réseaux récurrents, ils ne traitent pas les éléments de la séquence de manière strictement séquentielle, mais s'appuient principalement sur des mécanismes d'attention, en particulier l'auto-attention, afin de modéliser les relations entre les différents éléments d'une entrée [9]. Grâce à leur capacité à apprendre des dépendances complexes et à être entraînés efficacement sur de grandes quantités de données, les Transformers ont ensuite été largement utilisés dans de nombreux domaines de l'apprentissage profond, notamment le traitement du langage naturel, la vision par ordinateur et les modèles multimodaux [4]. Comme l'illustre la figure 1.3, un encodeur Transformer peut être représenté de manière simplifiée par un mécanisme d'auto-attention multi-têtes suivi d'un réseau feed-forward.

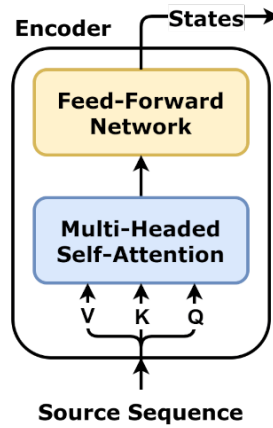


FIGURE 1.3 – Structure simplifiée d'un encodeur Transformer. Figure adaptée de dvgodoy, sous licence CC BY 4.0.

C. Vision Transformers

Les Vision Transformers, ou ViT, sont une adaptation de l'architecture Transformer au traitement des images. Au lieu de s'appuyer principalement sur des convolutions comme les CNN, une image est divisée en petits blocs appelés patches, par exemple de taille 16×16 . Ces patches sont ensuite transformés en vecteurs et traités comme une séquence d'éléments, de manière similaire aux tokens dans le traitement du langage naturel [10]. Grâce au mécanisme d'auto-attention, le modèle peut apprendre des relations globales entre différentes régions de l'image. Cette capacité rend les Vision Transformers particulièrement intéressants pour les tâches où la compréhension de la structure globale de l'image est importante, en complément des informations locales. Comme l'illustre la figure 1.4, les représentations des patches sont combinées à un encodage positionnel, puis transmises à un encodeur Transformer afin de produire la sortie du modèle.

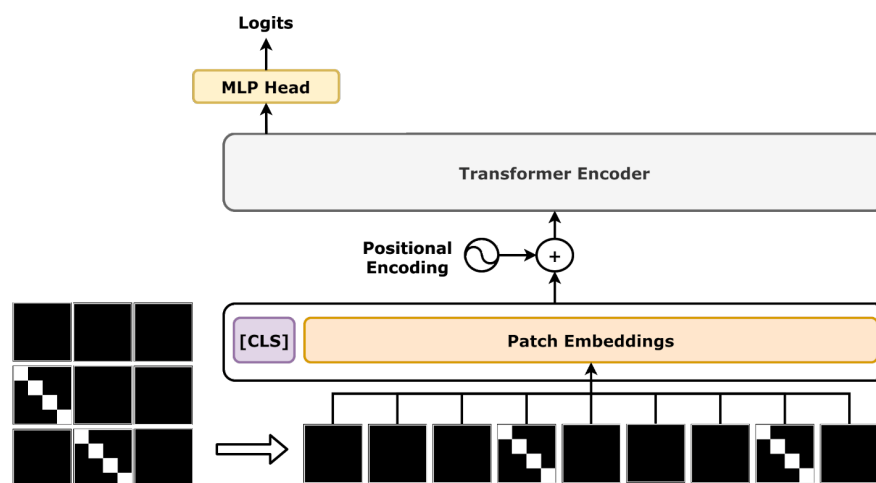


FIGURE 1.4 – Architecture générale d'un Vision Transformer. Figure adaptée de dvgodoy, sous licence CC BY 4.0.

1.4 Mécanismes d'attention

Les mécanismes d'attention constituent une composante importante des architectures modernes d'apprentissage profond, en particulier dans les modèles de type Transformer. Leur objectif est de permettre au modèle d'accorder une importance différente aux éléments d'une entrée selon leur pertinence pour la tâche à résoudre, au lieu de traiter toutes les informations de manière uniforme. Ce fonctionnement repose généralement sur trois éléments principaux : une requête, des clés et des valeurs. La requête représente l'information recherchée, les clés servent à mesurer la similarité avec cette requête, et les valeurs contiennent les informations à combiner [9].

Dans le cas de l'attention par produit scalaire mise à l'échelle, les requêtes, les clés et les valeurs sont regroupées respectivement dans les matrices Q , K et V . Les scores d'attention sont obtenus en calculant le produit entre les requêtes et les clés, puis en le normalisant par $\sqrt{d_k}$, où d_k désigne la dimension des clés. Un masque peut éventuellement être appliqué, notamment lorsque l'on souhaite empêcher le modèle d'accéder à certaines positions. La fonction softmax transforme ensuite, pour chaque requête, les scores obtenus en poids positifs compris entre 0 et 1, dont la somme est égale à 1. Les scores les plus élevés reçoivent ainsi les poids les plus importants, ce qui permet au modèle d'accorder davantage d'attention aux éléments considérés comme les plus pertinents. Ces poids sont finalement utilisés pour effectuer une combinaison pondérée des valeurs [9, 4]. Comme l'illustre la figure 1.5, ce mécanisme enchaîne le produit scalaire entre Q et K , la mise à l'échelle, l'application éventuelle d'un masque, la fonction softmax et la combinaison pondérée avec V .

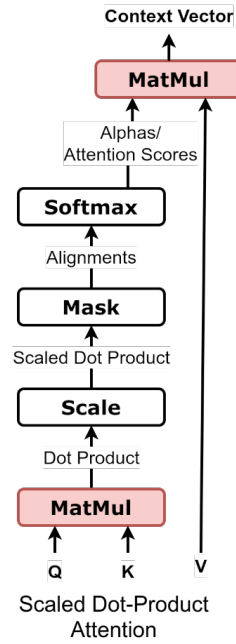


FIGURE 1.5 – Attention par produit scalaire mise à l'échelle. Figure adaptée de dvigodoy, sous licence CC BY 4.0.

Ce mécanisme peut être formalisé par l'équation suivante [9] :

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (1.1)$$

Cette équation montre que les similarités entre les requêtes Q et les clés K sont transformées en poids d'attention, puis utilisées pour pondérer les valeurs V . Le modèle peut ainsi privilégier les informations les plus pertinentes selon le contexte, y compris lorsqu'elles sont éloignées dans la séquence ou dans l'image [9, 4].

1.4.1 Self-attention

La self-attention, ou auto-attention, est un cas particulier du mécanisme d'attention dans lequel les requêtes, les clés et les valeurs proviennent de la même séquence. Elle permet donc à chaque élément d'une entrée de prendre en compte les autres éléments de cette même entrée afin de construire une représentation plus riche et contextualisée. Dans une phrase, par exemple, chaque mot peut être relié aux autres mots pour mieux comprendre son sens selon le contexte. Dans une image représentée sous forme de patches, chaque patch peut de la même manière interagir avec les autres régions de l'image [9]. Cette capacité à modéliser les relations internes d'une séquence explique l'importance de la self-attention dans les architectures Transformer. Dans le traitement d'images, cette propriété permet notamment de mettre en relation des régions éloignées d'une même image, contrairement aux opérations strictement locales qui se limitent à un voisinage restreint.

1.4.2 Cross-attention

La cross-attention, ou attention croisée, est une variante du mécanisme d'attention dans laquelle les requêtes ne proviennent pas de la même source que les clés et les valeurs. Contrairement à la self-attention, où un élément interagit avec les autres éléments d'une même séquence, la cross-attention permet à une représentation de s'appuyer sur une autre représentation externe.

Dans une architecture Transformer de type encodeur-décodeur, l'encodeur reçoit les données d'entrée et les transforme en représentations internes contenant les informations importantes de cette entrée. Le décodeur utilise ensuite ces représentations afin de générer progressivement la sortie attendue. Lors de la cross-attention, le décodeur produit les requêtes, tandis que les clés et les valeurs proviennent des représentations calculées par l'encodeur. Ce mécanisme permet ainsi au décodeur de sélectionner, à chaque étape de la génération, les informations de l'entrée les plus pertinentes [9, 4].

Dans le contexte de la vectorisation d'images, l'encodeur peut extraire les caractéristiques visuelles de l'image matricielle, tandis que le décodeur génère les paramètres des primitives vectorielles. La cross-attention permet alors aux requêtes associées aux primitives à produire d'interagir avec les clés et les valeurs issues de l'image encodée. Les paramètres vectoriels prédits sont ainsi guidés par les informations visuelles les plus pertinentes, ce qui facilite la génération de primitives adaptées au contenu de l'entrée.

1.5 Concepts utilisés dans la vectorisation par apprentissage profond

La vectorisation d'images par apprentissage profond repose sur plusieurs concepts techniques permettant de passer d'une image matricielle, composée de pixels, à une représentation vectorielle décrite par des formes paramétriques. Contrairement aux méthodes classiques de traçage, les approches récentes cherchent à apprendre ou à optimiser automatiquement les paramètres des primitives vectorielles afin de reconstruire au mieux l'image d'entrée. Pour cela, elles utilisent notamment la décomposition de l'image en régions plus simples, le rendu différentiable pour relier les paramètres vectoriels à leur rendu matriciel, ainsi que des fonctions de perte permettant de mesurer l'écart entre l'image originale et l'image reconstruite. Certaines méthodes adoptent également une stratégie progressive de type *coarse-to-fine*, dans laquelle la structure globale de l'image est d'abord reconstruite avant d'ajouter les détails fins [11, 12].

1.5.1 Superpixels

Les superpixels sont des régions obtenues en regroupant des pixels voisins présentant des caractéristiques similaires, notamment en termes de couleur et de position spatiale. Ils permettent de remplacer une représentation pixel par pixel par une représentation plus compacte, composée de régions visuellement cohérentes. Parmi les méthodes les plus connues, l'algorithme SLIC (*Simple Linear Iterative Clustering*) adapte le principe du k-means afin de générer efficacement des superpixels. Cette méthode est appréciée pour sa simplicité, sa rapidité, sa faible consommation mémoire et sa capacité à produire des régions qui respectent bien les contours de l'image [13]. Dans le cadre de la vectorisation d'images, les superpixels sont utiles car ils divisent une image complexe en sous-régions plus simples à traiter. SuperSVG exploite précisément cette idée en décomposant l'image d'entrée en superpixels contenant des pixels de couleurs et de textures similaires, puis en vectorisant ces régions de manière progressive [12].

1.5.2 Rendu différentiable

Le rendu différentiable est une technique qui permet de relier une représentation vectorielle à son rendu matriciel tout en conservant la possibilité de calculer des gradients. Dans le contexte de la vectorisation d'images, les paramètres des primitives vectorielles, comme les positions des points de contrôle, les couleurs ou l'épaisseur des traits, sont d'abord rendus sous forme d'image raster. L'image obtenue est ensuite comparée à l'image cible à l'aide d'une fonction de perte, puis les gradients de cette perte sont rétropropagés vers les paramètres vectoriels afin de les optimiser [11]. Cette approche permet donc d'utiliser des méthodes d'optimisation ou d'apprentissage profond pour ajuster automatiquement les formes vectorielles à partir d'une supervision dans le domaine des pixels. Elle est notamment utilisée dans des méthodes comme LIVE, qui optimise progressivement des chemins de Bézier dans un cadre de rendu entièrement différentiable afin de produire des SVG organisés en couches [14].

1.5.3 Fonctions de perte

Les fonctions de perte jouent un rôle central dans la vectorisation d'images par apprentissage profond, car elles permettent de mesurer l'écart entre l'image matricielle d'origine et l'image reconstruite à partir des primitives vectorielles. Dans les approches utilisant un rendu différentiable, les chemins SVG prédits ou optimisés sont d'abord rasterisés, puis comparés à l'image cible à l'aide d'une fonction de perte calculée dans l'espace des pixels. Une perte simple, comme la MSE, peut être utilisée pour minimiser la différence entre les deux images, mais elle ne suffit pas toujours à préserver correctement la structure, les contours ou la couleur des objets. Par exemple, LIVE montre qu'une

optimisation fondée uniquement sur la MSE peut conduire à un effet de couleur moyenne, puis propose une perte UDF afin de donner plus d'importance aux pixels proches des contours, ainsi qu'une perte Xing pour limiter les auto-intersections des courbes [14]. Dans SuperSVG, les auteurs combinent plusieurs pertes adaptées à la vectorisation par superpixels, notamment une perte de reconstruction normalisée, une perte de frontière pour empêcher les chemins de dépasser les limites du superpixel, et une perte Dynamic Path Warping permettant au modèle de raffinement d'hériter des connaissances du modèle grossier [12].

1.5.4 Apprentissage coarse-to-fine

L'apprentissage *coarse-to-fine* désigne une stratégie progressive dans laquelle le modèle commence par reconstruire les éléments globaux ou principaux d'une image, avant d'ajouter progressivement des détails plus fins. Dans le contexte de la vectorisation d'images, cette approche est particulièrement utile car une image complexe est difficile à convertir directement en une représentation SVG précise. En procédant par étapes, le modèle peut d'abord approximer la structure générale, puis améliorer localement la reconstruction. SuperSVG adopte cette logique à travers un cadre d'apprentissage en deux étapes : un modèle grossier prédit d'abord des chemins SVG pour reconstruire la structure principale du superpixel, puis un modèle de raffinement ajoute d'autres chemins afin d'enrichir les détails, en étant guidé par les chemins prédits à l'étape précédente [12]. Une idée similaire apparaît dans LIVE, où l'image est reconstruite progressivement en ajoutant et en optimisant de nouveaux chemins de Bézier dans une représentation en couches, ce qui permet d'obtenir un SVG plus compact et plus cohérent avec la structure visuelle de l'image [14].

1.6 Évaluation des résultats

L'évaluation d'une méthode de vectorisation consiste généralement à rendre le SVG généré sous forme matricielle, puis à comparer ce rendu à l'image d'entrée. Cette section présente les principales métriques utilisées dans nos expérimentations : la MSE, le PSNR, la SSIM, la LPIPS et le temps d'inférence.

1.6.1 MSE

La MSE (*Mean Squared Error*) mesure l'erreur moyenne entre les pixels de l'image originale I et ceux de l'image reconstruite $\hat{I}$:

$$\text{MSE}(I, \hat{I}) = \frac{1}{HWC} \sum_{i=1}^H \sum_{j=1}^W \sum_{c=1}^C (I_{i,j,c} - \hat{I}_{i,j,c})^2 \quad (1.2)$$

Dans l'équation (1.2), H et W représentent les dimensions de l'image et C son nombre de canaux. Une valeur faible indique une reconstruction plus fidèle au niveau des pixels. Cette métrique reste toutefois limitée, car elle ne correspond pas toujours à la qualité visuelle perçue [14, 12].

1.6.2 PSNR

Le PSNR (*Peak Signal-to-Noise Ratio*) est une métrique dérivée de la MSE. Pour des images dont les valeurs sont normalisées entre 0 et 1, il est défini par :

$$\text{PSNR}(I, \hat{I}) = 10 \log_{10} \left(\frac{1}{\text{MSE}(I, \hat{I})} \right) \quad (1.3)$$

Contrairement à la MSE, une valeur élevée du PSNR indique une meilleure fidélité de reconstruction. Cette métrique est principalement utilisée pour évaluer les différences pixel par pixel entre l'image cible et le rendu du SVG [11, 12].

1.6.3 SSIM

La SSIM (*Structural Similarity Index Measure*) mesure la similarité structurelle entre deux images en prenant en compte leur luminance, leur contraste et leur structure locale [15]. Elle est définie par :

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (1.4)$$

Dans l'équation (1.4), μ_x et μ_y désignent les moyennes des deux fenêtres comparées, σ_x^2 et σ_y^2 leurs variances, et σ_{xy} leur covariance. Les constantes C_1 et C_2 assurent la stabilité du calcul. Une valeur élevée indique une meilleure préservation de la structure visuelle [12].

1.6.4 LPIPS

La LPIPS (*Learned Perceptual Image Patch Similarity*) compare deux images à partir de caractéristiques extraites par un réseau de neurones pré-entraîné, plutôt qu'à partir de leurs pixels uniquement [16]. Elle peut être exprimée sous la forme :

$$\text{LPIPS}(x, \hat{x}) = \sum_l \frac{1}{H_l W_l} \sum_{h,w} \|w_l \odot (\phi_l(x)_{h,w} - \phi_l(\hat{x})_{h,w})\|_2^2 \quad (1.5)$$

Dans l'équation (1.5), ϕ_l représente les caractéristiques extraites à la couche l du réseau, tandis que w_l désigne les poids associés à ces caractéristiques. Une faible valeur correspond à une meilleure similarité perceptuelle entre l'image d'entrée et le rendu SVG [12].

1.6.5 Temps d'inférence

Le temps d'inférence correspond à la durée nécessaire pour produire un SVG à partir d'une image d'entrée. Il permet d'évaluer l'efficacité pratique d'une méthode, indépendamment de sa qualité de reconstruction. Les méthodes reposant sur l'optimisation directe des paramètres vectoriels nécessitent généralement plusieurs itérations, tandis que les modèles entraînés peuvent prédire ces paramètres plus rapidement. Cette métrique est donc utilisée conjointement avec les mesures de qualité afin de comparer la précision et la rapidité des différentes approches [12].

1.7 Travaux connexes

La vectorisation automatique d'images a connu une évolution progressive, allant des approches algorithmiques classiques jusqu'aux travaux récents combinant apprentissage profond et rendu différentiable. Pour comparer ces travaux, il ne suffit pas de considérer uniquement la similarité visuelle entre l'image d'entrée et le SVG obtenu. Une approche de vectorisation doit également être évaluée selon plusieurs critères, notamment la fidélité visuelle, le temps de calcul, le nombre de chemins générés, la simplicité du SVG, son éditabilité, sa capacité de généralisation et le niveau de contrôle laissé à l'utilisateur [17]. Dans cette section, nous présentons donc les approches les plus pertinentes pour notre sujet, en montrant progressivement comment les limites des approches classiques et fondées sur l'optimisation ont conduit à des travaux plus récents comme SuperSVG.

1.7.1 Approches algorithmiques

Les approches algorithmiques reposent généralement sur l'analyse géométrique de l'image, comme l'extraction de contours, la détection de régions ou l'ajustement de courbes. Elles sont souvent rapides et ne nécessitent pas de données d'entraînement, mais restent limitées lorsque l'image contient des couleurs complexes, des textures ou un grand nombre de détails.

A. Potrace

Potrace est une méthode classique de vectorisation principalement conçue pour convertir des images binaires en contours vectoriels [18]. Son principe consiste à extraire les frontières entre les zones noires et blanches, à les approximer par des polygones, puis à transformer ces polygones en courbes de Bézier lisses. Cette méthode est simple, rapide et efficace pour des images comme les logos, les textes ou les dessins en noir et blanc. Cependant, elle dépend fortement de la qualité de la binarisation et reste peu adaptée aux images complexes, colorées ou texturées. Potrace constitue donc une

référence importante pour les approches classiques, mais montre aussi les limites des méthodes purement algorithmiques face à une vectorisation plus générale.

1.7.2 Approches basées sur l'optimisation

Les approches basées sur l'optimisation cherchent à ajuster directement les paramètres des primitives vectorielles afin que le rendu obtenu soit le plus proche possible de l'image d'entrée. Avec le rendu différentiable, il devient possible de calculer des gradients entre l'image rendue et l'image cible, puis de les utiliser pour modifier les courbes, les couleurs ou l'opacité des chemins SVG.

A. DiffVG

DiffVG propose un rasteriseur différentiable permettant de relier le domaine vectoriel au domaine matriciel [11]. Grâce à ce mécanisme, un ensemble de courbes ou de formes vectorielles peut être rendu sous forme d'image, comparé à une image cible, puis optimisé par rétropropagation. Cette approche est importante car elle permet d'utiliser une supervision uniquement matricielle, sans avoir besoin d'un SVG de référence. Elle est donc devenue une base technique pour plusieurs méthodes récentes. Cependant, DiffVG reste coûteux lorsqu'il faut optimiser beaucoup de chemins ou effectuer un grand nombre d'itérations. De plus, le nombre de primitives doit souvent être fixé à l'avance, ce qui limite le contrôle automatique de la complexité du SVG.

B. LIVE

LIVE, pour *Layer-wise Image Vectorization*, améliore les approches d'optimisation en introduisant une reconstruction progressive par couches [14]. La méthode ajoute progressivement des chemins de Bézier, puis les optimise afin de reconstruire l'image de manière *coarse-to-fine*. Cette stratégie permet d'obtenir des SVG plus structurés, plus compacts et plus éditables que ceux produits par une optimisation naïve. LIVE introduit également des pertes spécifiques pour mieux préserver les contours et limiter les auto-intersections des chemins. Malgré ces avantages, la méthode reste lente, car chaque image nécessite une optimisation itérative. Le résultat dépend aussi du nombre de chemins choisi et de plusieurs hyperparamètres. LIVE montre donc l'intérêt d'une représentation vectorielle progressive et éditable, mais confirme aussi le besoin de méthodes plus rapides.

1.7.3 Approches basées sur l'apprentissage profond

Les approches basées sur l'apprentissage profond cherchent à apprendre directement la relation entre une image matricielle et une représentation vectorielle. Contrairement

aux méthodes d'optimisation qui ajustent les chemins pour chaque image, ces méthodes entraînent un modèle capable de prédire les paramètres vectoriels à partir de l'image d'entrée, ce qui permet généralement une inférence plus rapide.

A. Im2Vec

Im2Vec propose une méthode de génération de graphiques vectoriels sans supervision vectorielle directe [19]. Contrairement aux approches qui nécessitent des fichiers SVG annotés, le modèle apprend uniquement à partir d'images matricielles. Son architecture suit un schéma encodeur-décodeur : l'image d'entrée est d'abord encodée dans un espace latent, puis ce code latent est utilisé pour générer une représentation vectorielle composée de plusieurs chemins fermés. Pour gérer les formes constituées de plusieurs composants, Im2Vec utilise un module récurrent de type RNN/LSTM qui produit un code latent propre à chaque chemin. Chaque chemin est ensuite décodé sous forme de courbes de Bézier à l'aide d'un décodeur fondé sur des convolutions 1D circulaires, ce qui permet de modéliser des formes fermées comme des déformations d'un cercle.

Le résultat vectoriel généré est ensuite rendu sous forme matricielle à l'aide d'un rendu différentiable, puis comparé à l'image cible afin d'entraîner le modèle sans supervision SVG explicite. Cette approche est intéressante car elle évite le besoin de jeux de données vectoriels annotés, qui sont difficiles à obtenir et souvent ambigus, puisqu'une même image peut être représentée par plusieurs structures vectorielles différentes. Cependant, Im2Vec reste surtout adapté à des images relativement simples ou appartenant à des domaines limités, comme les icônes, les caractères ou les emojis. Il montre donc l'intérêt de l'apprentissage profond pour la génération et la vectorisation de formes paramétriques, mais reste limité pour traiter des images complexes et variées.

B. SuperSVG

SuperSVG est la méthode la plus proche de notre thématique et constitue une référence importante dans les travaux récents de vectorisation image-to-SVG [12]. Son principe consiste d'abord à décomposer l'image d'entrée en superpixels, afin de réduire la difficulté du problème en travaillant sur des régions plus homogènes en couleur et en texture. Chaque superpixel est ensuite vectorisé séparément par un modèle d'apprentissage profond chargé de prédire les paramètres des chemins SVG correspondants.

L'architecture de SuperSVG repose sur un modèle en deux étapes. Dans la première étape, dite coarse-stage, un encodeur Vision Transformer extrait les caractéristiques visuelles du superpixel. Ces caractéristiques sont ensuite combinées avec des requêtes apprenables associées aux chemins SVG à l'aide d'un module de cross-attention. Un module de self-attention traite ensuite les représentations intermédiaires afin de les projeter

vers l'espace des paramètres vectoriels. Cette première étape permet de reconstruire la structure principale du superpixel. Dans la seconde étape, dite refinement-stage, un second modèle prend en entrée le rendu obtenu à partir des chemins grossiers ainsi que le superpixel cible, puis prédit des chemins supplémentaires destinés à enrichir les détails. Les sorties des deux étapes sont finalement combinées pour produire le SVG final.

SuperSVG combine ainsi plusieurs idées importantes des travaux précédents : le rendu différentiable hérité de DiffVG, la reconstruction progressive proche de LIVE, et la prédiction directe de chemins SVG par apprentissage profond. Cette architecture lui permet d'obtenir un meilleur compromis entre qualité de reconstruction et temps de calcul, notamment par rapport aux méthodes purement basées sur l'optimisation. Néanmoins, ses résultats dépendent encore de la qualité de la décomposition en superpixels, du nombre de chemins utilisés et de la capacité du modèle à représenter correctement les détails fins de chaque région. Malgré ces limites, SuperSVG représente l'approche la plus pertinente pour notre état de l'art, car elle illustre l'évolution récente des méthodes de vectorisation vers des solutions plus rapides, plus précises et mieux adaptées aux images complexes.

1.8 Synthèse comparative des approches

Les travaux étudiés montrent que la vectorisation automatique d'images repose sur un compromis entre plusieurs critères : la fidélité visuelle, le temps de calcul, la simplicité du SVG généré, le nombre de chemins, l'éditabilité et la capacité de généralisation. Les méthodes algorithmiques privilégient généralement la rapidité et la simplicité, mais restent limitées aux images peu complexes. Les méthodes basées sur l'optimisation offrent une meilleure flexibilité grâce au rendu différentiable, mais leur coût augmente avec le nombre de chemins et d'itérations. Les approches basées sur l'apprentissage profond cherchent à dépasser cette limite en apprenant à prédire directement les paramètres vectoriels.

TABLE 1.1 – Synthèse comparative des principales approches de vectorisation étudiées.

Approche	Principe	Points forts	Limites	Apport
Potrace	Tracé de contours binaires et approximation par courbes de Bézier.	Rapide, simple et sans apprentissage.	Limité aux images complexes, colorées ou texturées; dépend de la binarisation.	Base des approches algorithmiques classiques.

Approche	Principe	Points forts	Limites	Apport
DiffVG	Optimisation de primitives SVG via un rendu différentiable.	Permet une supervision matricielle et la rétropropagation vers les paramètres SVG.	Coûteux, sensible à l'initialisation et au nombre de chemins fixé.	Introduit le rendu différentiable comme outil central.
LIVE	Ajout progressif de chemins de Bézier selon une logique couche par couche.	Produit des SVG plus structurés, compacts et éditables.	Temps de calcul élevé ; dépend des chemins et des hyperparamètres.	Montre l'intérêt d'une reconstruction progressive.
Im2Vec	Prédiction de formes vectorielles à partir d'images matricielles.	Évite la supervision SVG directe et apprend un espace latent.	Surtout adapté aux images simples comme les icônes, caractères ou emojis.	Introduit l'apprentissage profond pour prédire des primitives vectorielles.
SuperSVG	Vectorisation par superpixels avec reconstruction <i>coarse-to-fine</i> .	Bon compromis entre qualité, vitesse et traitement d'images complexes.	Dépend de la décomposition en superpixels et du nombre de chemins.	Approche récente la plus proche de notre problématique.

Cette comparaison montre que SuperSVG constitue l'approche la plus complète parmi les travaux étudiés. Elle reprend le rendu différentiable introduit par DiffVG, la logique progressive mise en avant par LIVE, et l'efficacité des modèles d'apprentissage profond. Cependant, la vectorisation automatique reste confrontée à plusieurs difficultés, notamment le contrôle de la complexité du SVG, la fidélité visuelle, le temps de calcul et la généralisation à des images variées.

1.9 Conclusion

Dans ce chapitre, nous avons présenté les notions essentielles liées à la vectorisation d'images, puis étudié les principales familles de méthodes existantes. L'analyse des travaux connexes montre une évolution progressive des approches algorithmiques vers les méthodes d'optimisation différentiable, puis vers les modèles basés sur l'apprentissage profond.

La suite du mémoire présentera l'approche proposée pour répondre à cette problématique, ainsi que les choix de conception retenus pour la mise en place du système de vectorisation.

Chapitre 2

Approche Proposée

2.1 Introduction

2.2 Dataset et prétraitement

2.2.1 Le prétraitement des données

- A. Former les paires d'images
- B. Augmentation de données

2.3 Méthode proposée

2.3.1 Définir les hyperparamètres évalués par les métaheuristiques

- A. L'Algorithme Génétique
- B. Les paramètres à optimiser
- C. Évaluation d'une solution

2.3.2 Architecture

- A. Sur-échantillonnage (Upsampling)
- B. Sous-échantillonnage (Downsampling)

2.4 Conclusion

Chapitre 3

Expérimentations et Résultats

3.1 Introduction

3.2 Environnement de développement

3.2.1 Hardware

TABLE 3.1 – Spécifications de la machine Kaggle

CPU	Mémoire	GPU
1,5 GHz	13 Go RAM + 73,1 Go HDD	Nvidia P100 — 16 Go VRAM

3.2.2 Software

3.3 Dataset utilisé

3.4 Prétraitement

3.5 Évaluation

3.5.1 Évaluation quantitative

3.5.2 Fonction de perte

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (3.1)$$

3.6 Paramétrage de l’Algorithme Génétique

- Taille de population : 10
- Nombre de générations : 6
- Probabilité de mutation : 0,5
- Fonction objective : PSNR

3.7 Sous-échantillonnage

3.7.1 Hyperparamètres avec la métaheuristique (GA)

TABLE 3.2 – Valeurs du PSNR en fonction du nombre de générations

Générations	PSNR (dB)
3	50,079
6	50,219

- Batch size : 19 Kernel size : 3×3
- Epochs : 27 Nombre de filtres : 64
- Nombre de couches : 5
- Learning rate : Adam Optimizer ($10^{-4} \rightarrow 10^{-5}$, limite 5000)

3.7.2 Résultats

A. Résultats quantitatifs

TABLE 3.3 – Résultats du downsampling (Set5) — PSNR / SSIM

Dataset	Facteur	Notre Modèle	Bicubique
Set5	$\times 2$	45,61 / 0,99	36,17 / 0,97
	$\times 3$	44,80 / 0,99	30,35 / 0,94
	$\times 4$	33,85 / 0,97	26,80 / 0,91

B. Résultats qualitatifs

3.8 Sur-échantillonnage

3.8.1 Hyperparamètres

- Batch size : 16 Kernel size : 3×3
- Epochs : 250 Nombre de filtres : 128
- Nombre de blocs résiduels : 12
- Learning rate : Adam Optimizer ($10^{-4} \rightarrow 10^{-5}$, limite 5000)

3.8.2 Résultats

A. Résultats quantitatifs

TABLE 3.4 – Comparaison quantitative sur Set5 (PSNR en dB)

Test set	Facteur	Bicubic	SRGAN	DRDN	EDSAN	CAR	SRCNN	Notre Modèle
Set5	×2	33,66	–	32,95	35,25	34,38	36,66	36,71
	×3	30,39	–	29,18	31,48	–	32,75	32,20
	×4	28,42	29,40	27,35	29,80	28,42	30,49	31,498

B. Résultats qualitatifs

3.9 Conclusion

Conclusion Générale

Au cours de ce projet de fin d'étude, nous avons exploré divers aspects de l'apprentissage profond et de la vision par ordinateur pour résoudre le problème du redimensionnement des images.

Bibliographie

- [1] Rafael C. GONZALEZ et Richard E. WOODS. *Digital Image Processing*. 4th. Pearson, 2018. ISBN : 978-1-292-22304-9.
- [2] James D. FOLEY et al. *Computer Graphics : Principles and Practice*. 2nd. Addison-Wesley Professional, 1996, p. 8.
- [3] W3C. *Scalable Vector Graphics (SVG) 1.1 (Second Edition)*. W3C Recommendation. World Wide Web Consortium, 2011. URL : <https://www.w3.org/TR/SVG11/>.
- [4] Aurélien GÉRON. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3rd. O'Reilly Media, 2022. ISBN : 978-1-098-12597-4.
- [5] Tom M. MITCHELL. *Machine Learning*. McGraw-Hill, 1997, p. 2.
- [6] Christopher M. BISHOP. *Pattern Recognition and Machine Learning*. Springer, 2006, p. 3.
- [7] Longlong JING et Yingli TIAN. “Self-Supervised Visual Feature Learning with Deep Neural Networks : A Survey”. In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43.11 (2021), p. 4037-4058. DOI : 10.1109/TPAMI.2020.2992393.
- [8] Yann LECUN et al. “Backpropagation Applied to Handwritten Zip Code Recognition”. In : *Neural Computation* 1.4 (1989), p. 541-551. DOI : 10.1162/neco.1989.1.4.541.
- [9] Ashish VASWANI et al. “Attention Is All You Need”. In : *Advances in Neural Information Processing Systems*. T. 30. Curran Associates, Inc., 2017.
- [10] Alexey DOSOVITSKIY et al. “An Image Is Worth 16x16 Words : Transformers for Image Recognition at Scale”. In : *International Conference on Learning Representations*. 2021. URL : <https://arxiv.org/abs/2010.11929>.
- [11] Tzu-Mao LI et al. “Differentiable Vector Graphics Rasterization for Editing and Learning”. In : *ACM Transactions on Graphics* 39.6 (2020), 193 :1-193 :15. DOI : 10.1145/3414685.3417871.
- [12] Teng HU et al. “SuperSVG : Superpixel-based Scalable Vector Graphics Synthesis”. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, p. 24892-24901.

- [13] Radhakrishna ACHANTA et al. “SLIC Superpixels Compared to State-of-the-Art Superpixel Methods”. In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 34.11 (2012), p. 2274-2282. DOI : 10.1109/TPAMI.2012.120.
- [14] Xu MA et al. “Towards Layer-wise Image Vectorization”. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, p. 16314-16323. DOI : 10.1109/CVPR52688.2022.01583.
- [15] Zhou WANG et al. “Image Quality Assessment : From Error Visibility to Structural Similarity”. In : *IEEE Transactions on Image Processing* 13.4 (2004), p. 600-612. DOI : 10.1109/TIP.2003.819861.
- [16] Richard ZHANG et al. “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric”. In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2018, p. 586-595. DOI : 10.1109/CVPR.2018.00068.
- [17] Maria DZIUBA et al. *Image Vectorization : A Review*. 2023. arXiv : 2306.06441 [cs.CV].
- [18] Peter SELINGER. *Potrace : A Polygon-Based Tracing Algorithm*. Technical report. Sept. 2003. URL : <https://potrace.sourceforge.net/potrace.pdf>.
- [19] Pradyumna REDDY et al. “Im2Vec : Synthesizing Vector Graphics without Vector Supervision”. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2021, p. 7342-7351.